

LATENT.SPACE · AGENTIC SYSTEMS

本体回归

AI 智能体为何重启语义网

概率推理 × 确定性边界 · Richard MacManus · 2026.07.30

让智能体自由思考，但不能自由越界

概率系统

LLM

擅长语言、推理与生成
开放、灵活、偶尔跑偏

本体



逻辑护栏

确定性系统

规则

定义实体、关系与约束
可计算、可检查、可执行

神经符号 AI：把神经网络接入符号规则，让生成能力运行在可验证边界内。

定义

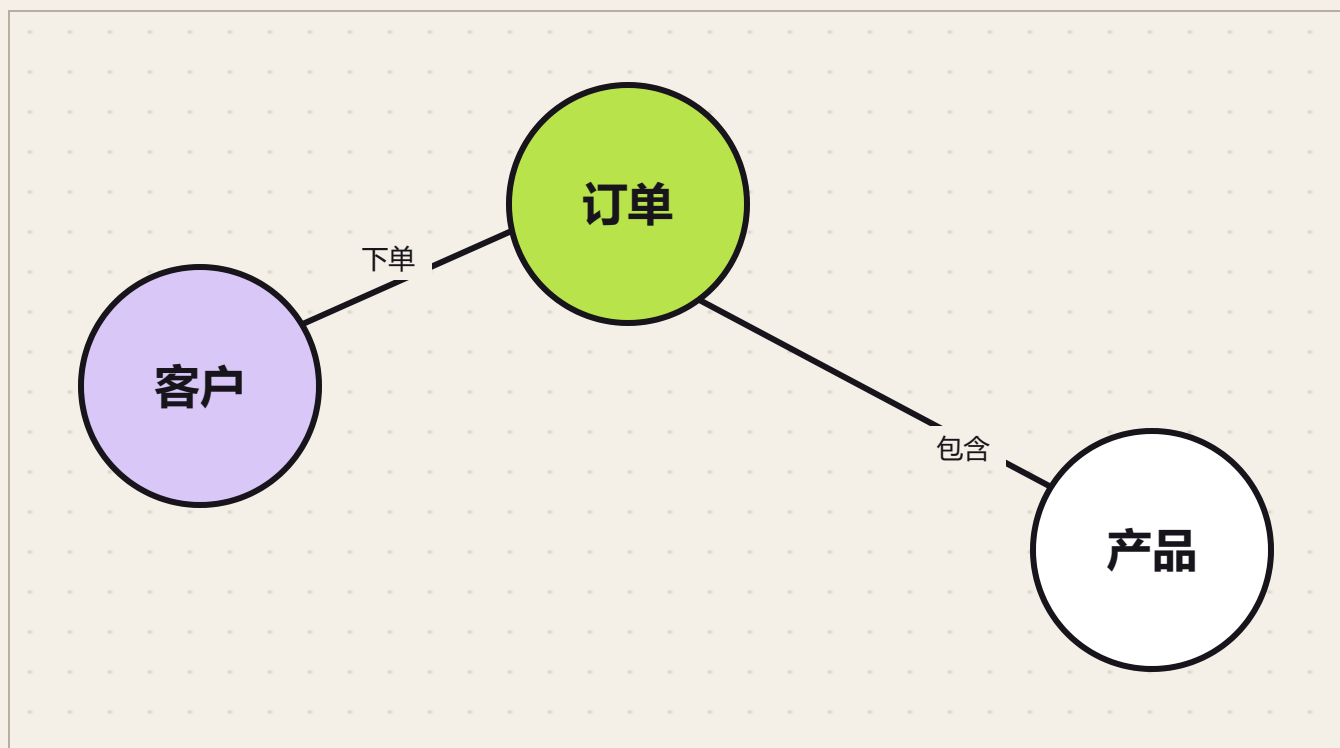
本体不是词典，而是可计算的世界模型

描述一个知识领域中的类、属性与关系。

“data as graphs”

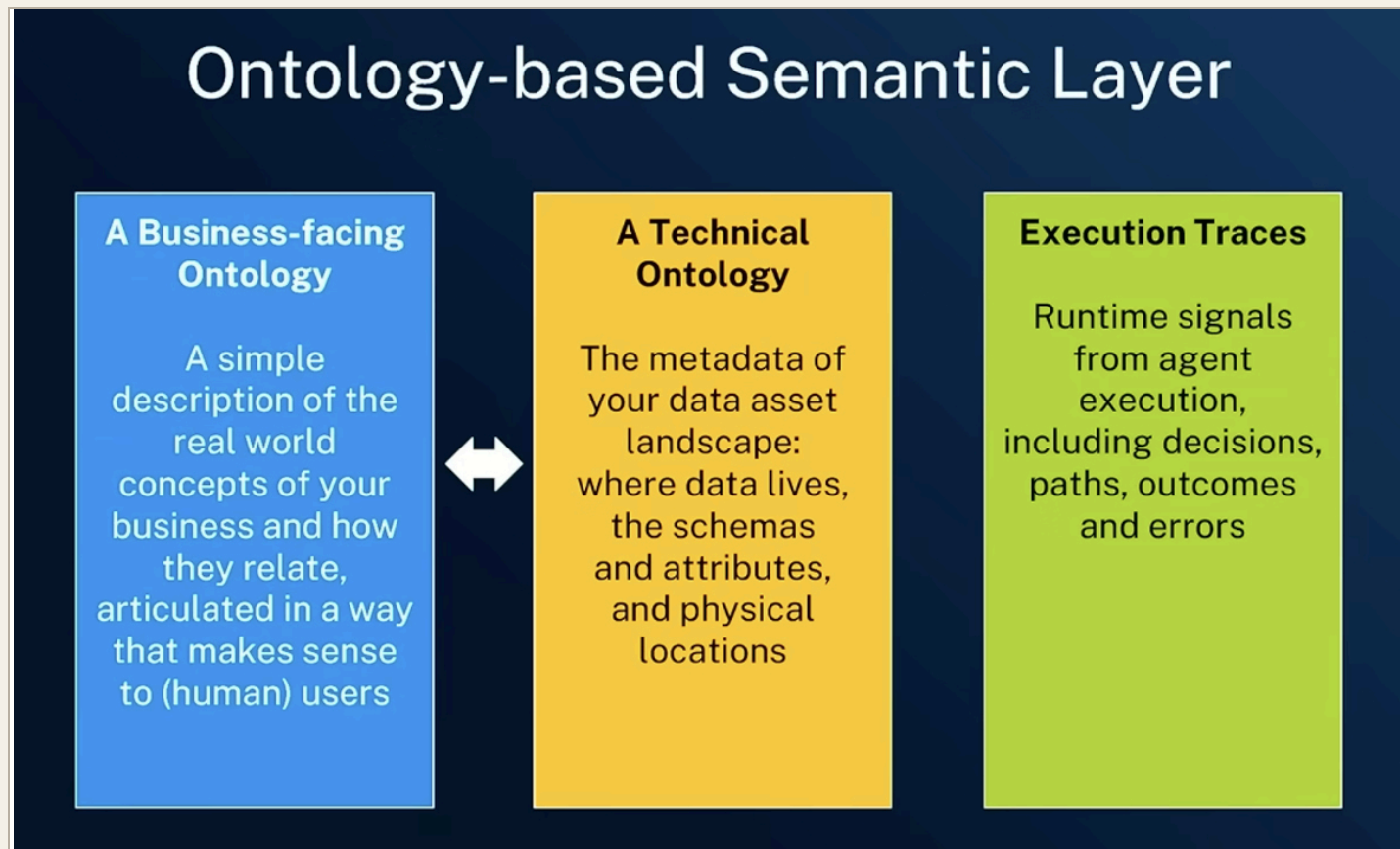
Frank Coyle

概念可追溯至亚里士多德；在 AI 历史中，它一直承担知识表示的角色。



企业智能体需要三种本体

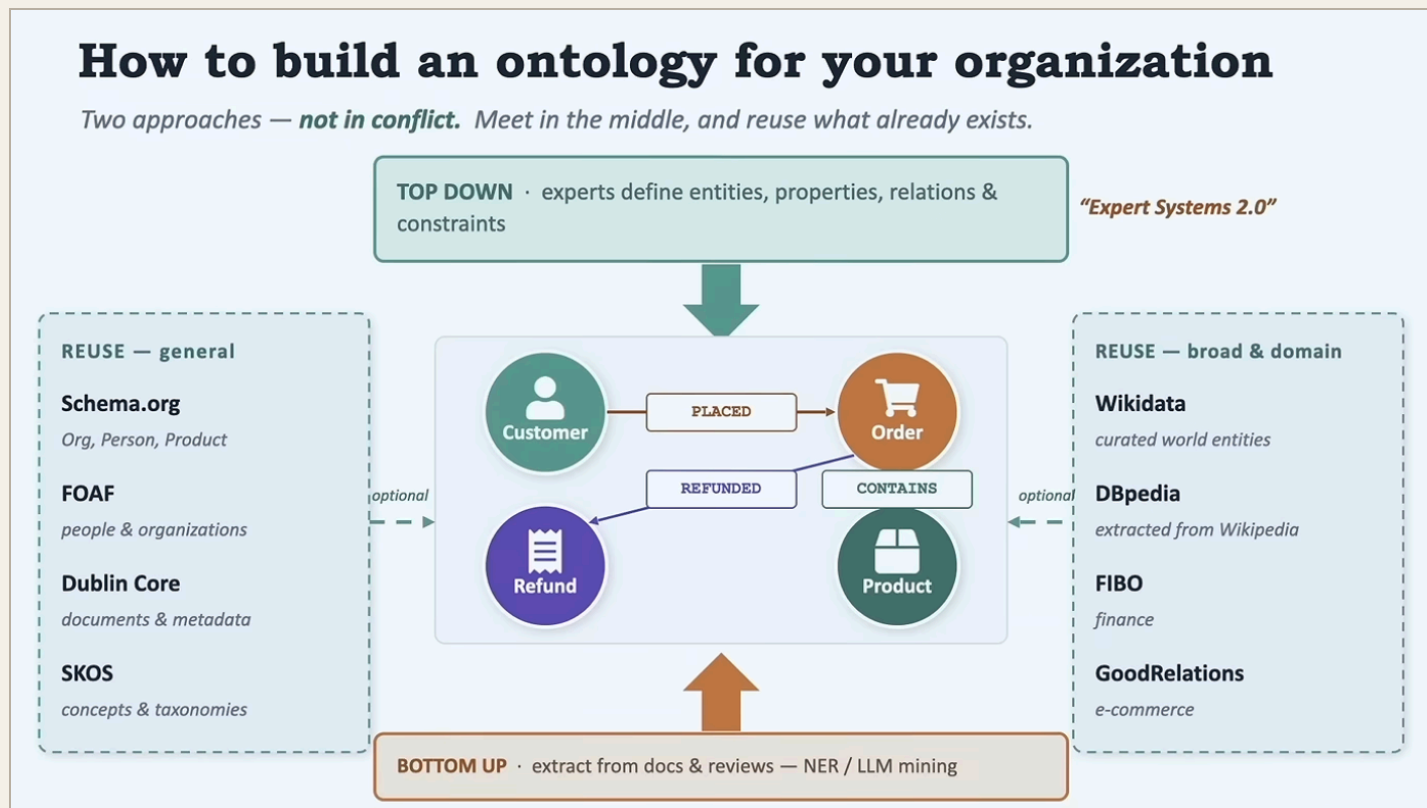
- 01 **业务本体**
组织里的关键概念
- 02 **技术本体**
数据源与资产的元数据
- 03 **执行轨迹**
智能体运行时信号



Neo4j: Ontology-based semantic layer

旧技术，新杠杆

不要从零发明：语义网已经写进模型



文章保留图：成熟 Web 本体与增强技术

Schema.org、FOAF、Dublin Core、RDFS、OWL 已存在多年。

关键优势

它们已经出现在 LLM 的训练数据里。

开发者可以直接提示模型采用标准，而不是重新定义整个领域。

生成之后，再由本体验证

LLM 推理

+

符号检查

=

可控智能体

工具执行并不是终点。结果回到本体层，检查类型、关系与规则，再决定继续、修正或停止。

AGENTIC LOOP ENGINEERING · the loop, and its two gates

A Claude agent loop: call a tool, check stop_reason, repeat

The model drives itself — each turn it asks for a tool or finishes. Two natural gates surround the tool call.

```
context_list = [user_task]    # the running memory

while True:
    resp = client.messages.create(
        model=MODEL, tools=TOOLS,
        messages=context_list)

    if resp.stop_reason == "tool_use":
        call = get_tool_call(resp) # refund(A-91)
        # ↳ model PREDICTS tool + args;
        #   our code names the block 'call'

        # ▶ GATE 1 - validate the INPUT here

        result = run_tool(call)    # tool runs

        # ▶ GATE 2 - validate the OUTPUT here

        context_list.append(
            tool_result(call.id, result))

    else: print(resp.text); break
```

TWO GATES, ONE PURE TOOL

context_list — the conversation so far; the loop grows it each turn.

GATE 1 — INPUT
before the tool runs.
validate the call's shape.

Pydantic

GATE 2 — OUTPUT
after the tool runs.
validate the result's coherence.

Ontology

Same pattern both times: *Pydantic guards what goes in, the ontology guards what comes out.* We'll fill both gates next.

Frank Coyle: ontology 在工具运行后验证推理

RDF memory: 记住事实，也记住关系



Kingsley Idehen: agent-rdf-memory 架构

语言提供表达力	本体提供可计算上下文	RDF把记忆变成可查询图
---------	------------	--------------

本体解决约束问题，却带来维护问题

+

获得

- 统一实体与关系
- 机器可执行的约束
- 跨智能体共享上下文
- 更容易审计与解释

价值

成本

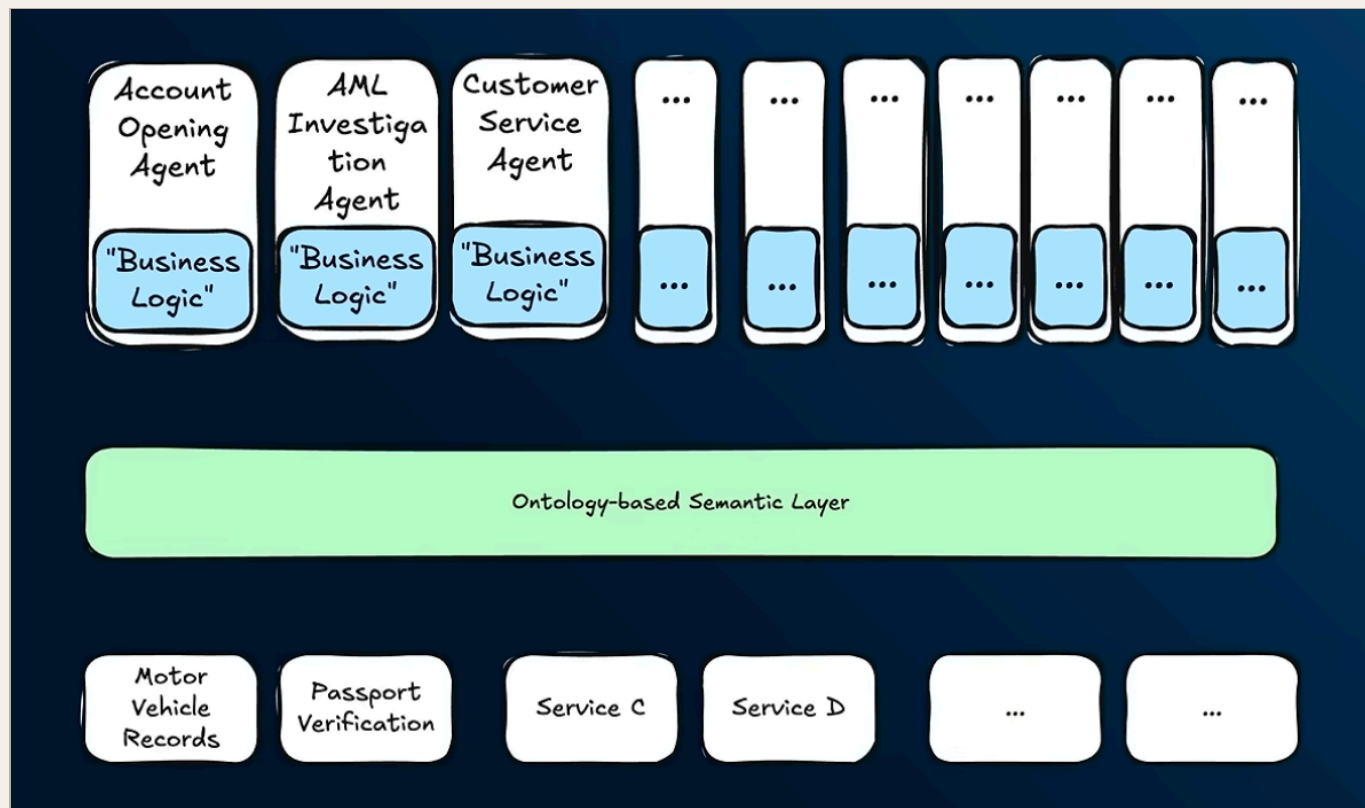
-

付出

- 持续维护定义
- 处理领域边界案例
- 同步业务变化
- 治理冲突与版本

Semantic Web 当年没有普及的老问题，今天并未自动消失。

从“厚智能体”转向“薄智能体”



Neo4j: thin agents on a smarter shared substrate

过去

厚智能体

每个智能体手工接线数据源，重复携带领域知识。

现在

薄智能体

把语义、元数据与轨迹放进共享底座。

给无限循环，套上有限规则



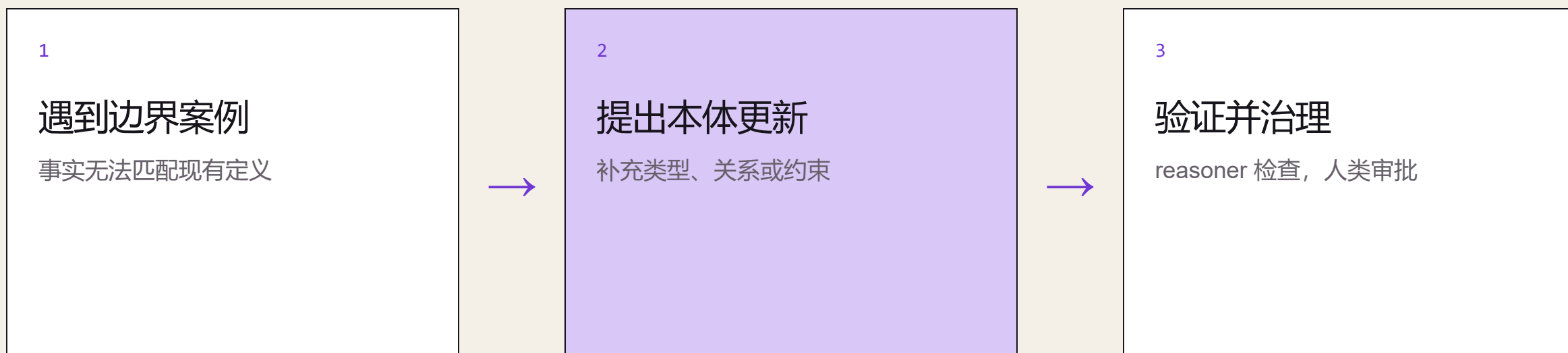
OWL catches what prose can't pin down

A sentence in a spec is a hope; an OWL axiom is a rule a machine enforces.

THE RULE, IN ENGLISH	THE OWL AXIOM	THE ERROR IT CATCHES
An order can be refunded at most once.	<code>:refunded a owl:FunctionalProperty</code>	<i>A second refund on the same order — the double-payout bug.</i>
A buyer and a support-desk employee are never the same account.	<code>:Customer owl:disjointWith :SupportRep</code>	<i>A payout sent to the support desk instead of the buyer.</i>
Order status is one of a fixed set — not free text.	<code>:Status owl:oneOf (paid shipped refunded)</code>	<i>A made-up value like “probably shipped” no tool can read.</i>
A refund must point back to an order that exists.	<code>:refunded rdfs:domain :Order</code>	<i>A refund attached to the wrong kind of thing — a bad call.</i>

语言可能含糊；OWL axiom 是机器强制执行的规则

让智能体在工作中维护本体



维护问题并没有消失，但它的性质变了：从离线文档工程，变成智能体运行的一部分。

一个可执行的本体护栏栈



建议：先从一个高风险决策点与一组小型规则开始，而不是建造“全企业本体”。

结论

更强的智能体， 需要更清晰的边界

01

LLM 提供
概率能力

02

本体提供
确定性语境

03

人类提供
治理与责任

来源: Latent.Space · “Ontologies Are So Back”